PropOS 楼层结构标注器 v1.5 — 验证 / 部署操作指南

> 适用模块：M1 资产 — Floor Structure Annotator v1.5
>
> 涉及组件：`backend/`（Dart + Shelf）、`admin/`（Vue 3）、`scripts/floor_map/` 检测器
>
> 关键变更：新增 migration `027_create_floor_maps.sql`、4 个 REST 端点、admin 标注 UI、检测器 `--extract-structures --db-url`

本文档提供两条平行的验证路径：

- **Track A — 本地验证**：在开发机一站式跑通端到端，不依赖远程基础设施。
- **Track B — 远程服务器部署验证**：将代码推到生产服务器（`propos-server` / `111.230.112.246`），按线上拓扑验证。

> ⚠️ 部署脚本变化提示：`backend/deploy.sh` 与 `admin/deploy.sh` 本身无需修改（它们只负责镜像/静态资源更新），但本次新增了 migration 027；为避免与 `setup_server.sh` 全量流程耦合，本次新增辅助脚本：
>
> - scripts/apply_migration.sh：单文件 migration 应用器，支持 `--target local|remote`，幂等，复用现有 `schema_migrations` / `_schema_migrations` 追踪表。

---

0. 前置检查（两条路径共用）

0.1 代码完整性

```
# 关键文件必须存在
ls backend/migrations/027_create_floor_maps.sql
ls backend/lib/modules/assets/models/floor_map.dart
ls backend/lib/modules/assets/repositories/floor_map_repository.dart
ls admin/src/stores/floorStructuresStore.ts
ls admin/src/views/assets/floor-structures/AnnotatorView.vue
ls scripts/floor_map/layer_constants.py
ls scripts/apply_migration.sh
```

0.2 Python 检测器依赖（两条路径都要写库）

检测器使用 `--db-url` 参数时依赖 `psycopg`（v3），但当前未在锁定依赖中：

```
source .venv/bin/activate
pip install ezdxf lxml Pillow psycopg[binary]
python -c "import ezdxf, psycopg; print('ok')"
```

0.3 业务前置数据

检测器按 `floors.floor_name` 关联到 `floor_maps`，因此 buildings + floors 必须已 seed：

```
# 本地:
```

```
PGPASSWORD=ChangeMe_2026! psql -h localhost -U propos -d propos_dev -f scripts/
seed_buildings_24f.sql
# 远程：参见 Track B
```

Track A — 本地验证

A1. 数据库初始化（首次）

```
# 走完整初始化流程：建库 + 应用所有 migrations + 注入 020/023 种子
PGUSER=$(whoami) bash scripts/init_local_postgres.sh
```

该脚本会自动包含 027（按文件名顺序），并写入 `schema_migrations` 表。

A2. 数据库已存在 → 单独应用 027

若你早先已 init 过本地库，只需追加 027：

```
PGPASSWORD=ChangeMe_2026! \
  bash scripts/apply_migration.sh --target local \
  --file backend/migrations/027_create_floor_maps.sql
```

预期输出： `[本地] ✓ 完成：027_create_floor_maps.sql`

A3. 验证 schema 正确

```
PGPASSWORD=ChangeMe_2026! psql -h localhost -U propos -d propos_dev <<'SQL'
\d floor_maps
SELECT column_name FROM information_schema.columns
 WHERE table_name='floors' AND column_name IN
('render_mode','floor_map_schema_version','floor_map_updated_at');
SELECT filename FROM schema_migrations WHERE filename LIKE '027%';
SQL
```

应看到 `floor_maps` 表结构， `floors` 含三个新列， `schema_migrations` 含 027 行。

A4. seed 楼栋/楼层

```
PGPASSWORD=ChangeMe_2026! psql -h localhost -U propos -d propos_dev \
  -f scripts/seed_buildings_24f.sql
```

A5. 跑检测器并写库

```
source .venv/bin/activate
export DATABASE_URL='postgresql://propos:ChangeMe_2026!@localhost:5432/propos_dev'

# 单楼栋一气呵成：DXF→SVG → 候选写库
python3 scripts/split_dxf_by_floor.py \
  cad_intermediate/building_a/A座.dxf \
```

```
  --output-dir cad_intermediate/building_a/floors \
  --extract-structures \
  --db-url "$DATABASE_URL"
```

预期输出包含每层 `[Fxx] Stage 7 DB: 已 UPSERT 到 floor_maps (floor_id=...)` 。

A6. 启动后端

```
cd backend
dart pub get
dart run bin/server.dart
# 监听 http://localhost:8080
```

A7. 启动 admin

```
cd admin
pnpm install
pnpm dev
# 浏览器打开 http://localhost:5173
```

A8. UI 操作走查

登录后 → 资产 → 楼栋详情 → 任选一层:

- [ ] 楼层卡片显示 `render_mode` 标签（默认 `semantic` ）
- [ ] 点击「结构标注」进入标注器（路由 `assets/buildings/:bid/floors/:fid/structures` ）
- [ ] 候选面板可加载该层 `candidates`
- [ ] Canvas 渲染 outline / structures / windows / north
- [ ] 撤销 / 重做（最多 20 步）正常
- [ ] 保存后 `Toolbar` 提示成功；ETag/If-Match 防并发覆盖
- [ ] 切换 `render_mode = vector` 后，列表标签同步更新

A9. 接口直查

```
TOKEN=...   # 从浏览器 devtools 取
FID=...     # 任意一层 id

curl -s -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/floors/$FID/structures | jq .

curl -s -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/floors/$FID/structures/candidates | jq '.data | keys'
```

Track B — 远程服务器部署验证

服务器拓扑（参考 `scripts/setup_server.sh` ）:

- `propos-postgres` （容器） — PostgreSQL 15
- `propos-backend` （容器） — `ccr.ccs.tencentyun.com/ephnic/propos_backend:latest` ，端口 8080
- `propos-nginx` （容器） — 反代 + 静态文件，端口 80
- 配置： `/opt/propos/.env` 、 `/opt/propos/nginx.conf` 、 `/opt/propos/admin-dist`
- 文件卷： `propos-uploads:/data/uploads`
- SSH 别名： `propos-server` ，TCR 凭据在项目根 `.deploy.env`

B1. 部署后端镜像

```
bash backend/deploy.sh
```

该脚本会：① 本地 docker build linux/amd64 → ② 推 TCR → ③ ssh 拉取 → ④ 重启 `propos-backend` 。
不会自动跑 migration。

B2. 应用 migration 027 到远程

```
bash scripts/apply_migration.sh --target remote \
  --file backend/migrations/027_create_floor_maps.sql
```

脚本行为：

1. `rsync` 该 SQL 到远程 `/tmp/`
2. `ssh propos-server` → `docker cp` 进 `propos-postgres` → `psql -f` 应用
3. 写入 `_schema_migrations` （含 file_hash 防篡改）

> 若之前已通过 `setup_server.sh` 跑过 027，脚本会输出 跳过（已应用） ，幂等。

B3. 验证远程 schema

```
ssh propos-server "docker exec propos-postgres psql -U propos -d propos -c \
  \"SELECT filename FROM _schema_migrations WHERE filename LIKE '027%';\""

ssh propos-server "docker exec propos-postgres psql -U propos -d propos -c '\d floor_maps'"
```

B4. 部署 admin 静态资源

```
bash admin/deploy.sh
```

该脚本会：① `pnpm build` （ `VITE_API_BASE_URL=''` ） → ② `rsync dist → /opt/propos/admin-dist` → ③ `rsync nginx.conf` → ④ 重启 `propos-nginx` 。
本次未引入新环境变量，无需修改。

B5. seed 楼栋/楼层（如果远程库尚未 seed）

```
rsync -az scripts/seed_buildings_24f.sql propos-server:/tmp/
ssh propos-server "docker cp /tmp/seed_buildings_24f.sql propos-postgres:/tmp/ && \
  docker exec propos-postgres psql -U propos -d propos -f /tmp/seed_buildings_24f.sql"
```

B6. 跑检测器（在本机执行，写远程库）

```
source .venv/bin/activate
# 远程 DB 端口需通过 SSH 隧道暴露，或临时放开安全组（仅限验证）
ssh -fNL 15432:localhost:5432 propos-server
export DATABASE_URL='postgresql://propos:<REMOTE_PG_PASSWORD>@localhost:15432/propos'

python3 scripts/split_dxf_by_floor.py \
  cad_intermediate/building_a/A座.dxf \
  --output-dir cad_intermediate/building_a/floors \
  --extract-structures \
  --db-url "$DATABASE_URL"

# 用完关闭隧道
pkill -f 'ssh -fNL 15432'
```

B7. UI 走查（生产）

浏览器访问 `http://111.230.112.246` → 登录 → 资产 → 楼栋详情 → 任选一层 → 完成 A8 中相同清单。

B8. 健康检查

```
ssh propos-server "docker logs --tail 100 propos-backend | grep -E 'floor_maps|ERROR' || true"
ssh propos-server "docker logs --tail 50 propos-nginx | tail -20"
curl -s http://111.230.112.246/api/health
```

验证检查清单（汇总）

维度	检查项	通过判据
Schema	`floor_maps` 表存在	`\d floor_maps` 输出 11 个字段
Schema	`floors` 新增 3 列	`render_mode/floor_map_schema_version/floor_map_updated_at` 可见
Schema	migration 已记录	`schema_migrations` / `_schema_migrations` 含 027 行
检测器	DB 写入	每层日志 `Stage 7 DB: 已 UPSERT`
检测器	命中率	业务楼层 100%，参考 `docs/report/floor_structure_detector_hit_rate.md` v3
API	GET /candidates	返回 `outline / structures / windows / north / candidates`
API	PUT /structures	第二次相同 ETag 提交返回 `409`（乐观锁）
API	PATCH /render-mode	DB 中 `floors.render_mode` 同步
UI	入口	楼栋详情页「结构标注」按钮可见且可点击
UI	撤销/重做	至少支持 20 步历史
UI	状态色	标签按 `render_mode` 区分 vector / semantic

回滚

后端镜像回滚

```
ssh propos-server "docker pull ccr.ccs.tencentyun.com/ephnic/propos_backend:<上一版本 tag> && \
  docker stop propos-backend && docker rm propos-backend && \
  docker run -d --name propos-backend --network propos-net -p 8080:8080 \
    --env-file /opt/propos/.env -v propos-uploads:/data/uploads \
    ccr.ccs.tencentyun.com/ephnic/propos_backend:<上一版本 tag>"
```

Migration 027 回滚（仅在确认必要时）

该 migration 没有提供 **027.undo.sql**。若必须回滚：

```
-- 备份后执行
DROP TABLE IF EXISTS floor_maps;
ALTER TABLE floors DROP COLUMN IF EXISTS render_mode;
ALTER TABLE floors DROP COLUMN IF EXISTS floor_map_schema_version;
ALTER TABLE floors DROP COLUMN IF EXISTS floor_map_updated_at;
DELETE FROM schema_migrations WHERE filename='027_create_floor_maps.sql';
-- 远程同步 _schema_migrations
```

admin 静态资源回滚

```
ssh propos-server "ls /opt/propos/admin-dist.bak.* 2>/dev/null"   # 之前部署若有备份
# 或重新 git checkout 上一版后再 bash admin/deploy.sh
```

已知限制 / 后续工作

编号	限制	处理建议
L1	多层合并标签（如 F12-F14）仅匹配单条 floor 行	由数据建模阶段决定：拆为 3 行 / 主-从关联 / 跳过
L2	`outline` 当前为凸包，含外凸装饰	后续可改为 alphashape 或人工修正
L3	psycopg 未进入 pinned requirements	待新增 `requirements.txt`（脚本侧）
L4	`setup_server.sh` 是初始化脚本，单独迁移走 `scripts/apply_migration.sh`	已在本文档说明

相关文档

- 实现计划：docs/plan/FLOOR_STRUCTURE_ANNOTATOR_IMPL_PLAN.md
- 命中率报告 v3：docs/report/floor_structure_detector_hit_rate.md
- 后端 PG 使用指南：docs/backend/guide/POSTGRES_USAGE_GUIDE.md
- API 契约 v1.7：docs/backend/API_CONTRACT_v1.7.md